

---

# CG2271 Real Time Operating Systems

---

## Lab Lecture 7 Real Time Operating Systems

---

Lab Lecture 7 – Real Time Operating Systems

# INTRODUCTION

# 1. Real-Time Operating Systems

- We are all familiar with operating systems:
  - ❑ Makes the hardware on our computers available to us:
    - Otherwise you will need to program your keyboard, screen, WiFi and everything else “bare-metal”!
  - ❑ Allows us to plug in hardware from different manufacturers without problem.
    - Achieved through “device drivers”, most are built into the OS.
    - Otherwise you will not be able to use even disk drives from different manufacturers!
  - ❑ Our main way to interact with your computers.
    - Through a part of the OS called the “shell”.



# 1. Real-Time Operating Systems

- A real-time operating system (RTOS) is very different from what we understand as an “operating system”.
  - ❑ Focus is on small size, high speed, reliability, and “guaranteed deadlines”.
  - ❑ No shell; your application starts the OS!
- Used in mission critical system:
  - ❑ Aircraft flight and engine controls.
  - ❑ Car engine and braking controls.
  - ❑ Industrial control systems.
  - ❑ ...



# 1. Real-Time Operating Systems

- ❑ Mostly, you still need to program the hardware yourself. But it gives you facilities for:
  - Managing multiple “tasks”.
    - ❑ A “task” is (almost) equivalent to a thread.  
Implemented as functions with infinite loops.
  - Providing synchronization and communications between tasks.
    - ❑ FIFO queues, priority queues, semaphores, monitors.
    - ❑ Also provided by normal OS, but much more critical in an RTOS.
- In this class we will use FreeRTOS.
  - ❑ Fully featured RTOS.
  - ❑ SafeRTOS is a safety-certified version of FreeRTOS



# 1. Real-Time Operating Systems

- One (of many) important difference between an RTOS and a “normal” OS.
  - ❑ In a normal OS, we start up the OS, and use the OS to start up the application, which then may fork other processes or spawn threads.
  - ❑ In an RTOS:
    - The application registers tasks that need to be run.
    - The application then starts the OS.
  - ❑ So in an RTOS it is the application that starts the OS, not the other way around.



---

Lab Lecture 7 – Real Time Operating Systems

# RTOS TASKS

## 2. RTOS Tasks

- FreeRTOS systems are centred upon “tasks” and interrupts.
  - ▣ Tasks are similar to POSIX threads
    - Every task is usually written as a function.
    - Just like pthreads, a single function can be used to spawn multiple threads.

```
int sum=0;
void *dosum(void *param) {
    printf("I am thread %d\n",
(int) param);
    sum++;
}

int main() {
    pthread_t tids[10];
    for(int i=0; i<10; i++) {
        // Create 10 threads
        pthread_create(&tids[i], NULL, dosum,
            (void *) i);
    }
    return 0;
}
```

## 2. RTOS Tasks

### ■ Example equivalent task in FreeRTOS:

```
int sum=0;
static void dosum(void *param) {
    while(1) {
        sum++;
        vTaskSuspend(NULL);
    }
}

int main() {
    char taskname[20];
    for(i=0; i<10; i++) {
        sprintf(taskname, "dosum_%d", i);
        xTaskCreate(dosum, taskname,
                    configMINIMAL_STACK_SIZE+100,
                    NULL, 1, (void *) i);
    }
}
```

### ■ Key differences:

- ❑ Tasks generally run in infinite loops. Call vTaskSuspend if the task should not loop anymore. Note: The “static” keyword is optional.
- ❑ Tasks have priorities at creation. Here all tasks have equal priority of 1 (smaller number = lower priority).

## 2. RTOS Tasks

- ARM Cortex M0+ version of FreeRTOS is pre-emptive by default - A higher priority task will pre-empt a lower-priority task when it becomes ready.
- However time-slicing is turned off by default. In the code below, one of the two tasks will not be able to run as they have equal priority and neither gives up CPU control.

```
static void task1(void *p) {  
    while(1) {  
        PRINTF("TASK 1\r\n");  
    }  
}  
  
static void task2(void *p) {  
    while(1) {  
        PRINTF("TASK 2\r\n");  
    }  
}  
  
int main() {  
    xTaskCreate(task1, "task1",  
                configMINIMAL_STACK_SIZE+100,  
                NULL, 1, NULL);  
  
    xTaskCreate(task2, "task2",  
                configMINIMAL_STACK_SIZE+100,  
                NULL, 1, NULL);  
}
```

## 2. RTOS Tasks

- A task needs to yield CPU (e.g. calling `vTaskDelay`) to allow another task at the same or lower priority level to run:

```
static void task1(void *p) {  
    while(1) {  
        PRINTF("TASK 1\r\n");  
        vTaskDelay(  
            pdMS_TO_TICKS(250));  
    }  
}  
  
static void task2(void *p) {  
    while(1) {  
        PRINTF("TASK 2\r\n");  
        vTaskDelay(  
            pdMS_TO_TICKS(250));  
    }  
}  
  
int main() {  
    xTaskCreate(task1, "task1",  
                configMINIMAL_STACK_SIZE+100,  
                NULL, 1, NULL);  
  
    xTaskCreate(task2, "task2",  
                configMINIMAL_STACK_SIZE+100,  
                NULL, 1, NULL);  
}
```

## 2. RTOS Tasks

### ■ What are tasks used for?

#### □ Controlling actuators.

- Generally each actuator will have its own task.
  - A buzzer task, and LCD task, a linear motor task, etc.
- However groups of actuators of one kind might be easier to control from a single task.
  - E.g. four motors of a robot should be controlled from a single task.



#### □ Polling sensors that do not support interrupts.

- Again general rule is one task per sensor.
- However, may make more sense for groups of the same type of sensor to be controlled by a single task.
  - E.g. polling a group of thermistors to measure temperatures around a device.



## 2. RTOS Tasks

- What are tasks used for?
  - ❑ Handling data from sensors that trigger interrupts.
    - E.g. radios, networking cards, microswitches.
  - ❑ Generally one task per sensor.
    - Again makes sense to process groups of microswitches together.
    - Generally though each serial or network input should have its own task.
  - ❑ Why have a task? Why not do everything in the ISR?



---

Lab Lecture 7 – Real Time Operating Systems

# INTEGRATING INTERRUPTS WITH TASKS

### 3. Integrating Interrupts with Tasks

- Tasks have priorities:

- More important tasks that should get precedence are given higher priorities.
  - In FreeRTOS, larger priority number == higher priority.
- Other tasks (e.g. tasks that clear up unused data in memory) are less important and run at lower priorities.

- Interrupts have priorities too.

- Recall: `NVIC_SetPriority` lets us assign four possible priority levels:
  - 00 (highest), 64, 128, 192 (lowest).

### 3. Integrating Interrupts with Tasks

- One key issue in designing ISRs:
  - ❑ A high priority ISR will block all other lower priority ISRs.
  - ❑ Worse: A high priority ISR will block all other ISRs of equal priority.
  - ❑ Worst: The lowest priority ISR will block the highest priority tasks.
    - This is because task priorities are managed by software (i.e. the RTOS).
    - ISRs are triggered directly by hardware and bypass the RTOS.
- Thus there are some important principles in designing ISRs.

### 3. Integrating Interrupts with Tasks

- Important rules for ISRs:

- ❑ An ISR must do the minimal so that it completes quickly.
  - E.g. read the data, start a task to handle the data.
  - The ISR should not do any processing.
- ❑ An ISR must not make any blocking calls.
  - E.g. wait for data.
- ❑ An ISR must not call functions that make blocking calls.
  - E.g. call a function that waits for data.
- ❑ An ISR must not call a function that is very slow.
  - E.g. sending data over a serial port.

- Following these principles reduce the risk of an ISR blocking an important task (or other important interrupts).

### 3. Integrating Interrupts with Tasks

#### ■ Binary Semaphores:

- ❑ An integer that holds either 0 or 1 (cannot hold any value  $>1$  or  $<0$ ).
- ❑ If a task “takes” a semaphore:
  - If the semaphore is 1, it decrements to 0 and exit.
  - If it is 0, the task blocks and enters into a blocked task queue.
- ❑ If a task “gives” or “signals” a semaphore:
  - If the semaphore is 0 and the blocked task queue is empty, increment the semaphore to 1. Exit.
  - If the semaphore is 0 and the blocked task queue has at least one task, choose one to unblock. Exit.
  - If the semaphore is 1, exit.

#### ■ Note: In FreeRTOS, binary semaphores start with an initial value of 0.

### 3. Integrating Interrupts with Tasks

- How do we use semaphores to interface between ISRs and tasks?
  - ❑ Create a binary semaphore initialized to 0 (default in FreeRTOS).
    - Task waits on the semaphore.
      - If the ISR has not yet triggered, the semaphore is 0 and the task blocks.
      - If the ISR has already been triggered, the semaphore is 1 and the task proceeds.
  - ❑ ISR gives (signals) on the semaphore.
    - If there is a task waiting, it is unblocked and allowed to run.
    - If no task is waiting, the semaphore is simply incremented to 1.

### 3. Integrating Interrupts with Tasks

- E.g. task waiting for a string coming over the UART:

```
SemaphoreHandle_t sema; // Semaphore

// Data received from UART
char recvBuffer[128];
void UART2_FLEXIO_IRQHandler() {
    static int currpos=0;
    char data;
    NVIC_ClearPendingIRQ(UART2_FLEXIO_IRQn);
    if(UART2->S1 & UART_S1_RDRF_MASK){
        data = UART2->D; // Read serial.
        recvBuffer[currpos++] = data;
        if(data == '\\0') {
            BaseType_t hpw;
            // End of string. Signal task.
            currpos = 0; // Reset position.
            xSemaphoreGiveFromISR(sema, &hpw);
            portYIELD_FROM_ISR(hpw);
        }
    }
}
```

```
static void uartTask(void *p) {
    while(1){
        // Wait on semaphore.

        if(xSemaphoreTake(sema,
            portMAX_DELAY)== pdTRUE){
            ... Code to handle data
                in recvBuffer ...
        }
    }
}

int main() {
    ... other code ...
    sema =
        xSemaphoreCreateBinary();
}
```

---

Lab Lecture 7 – Real Time Operating Systems

# THE FREERTOS INTERFACE

## 4. The FreeRTOS Interface

- This section looks at some useful FreeRTOS functions.

- Full reference manual:

[https://www.freertos.org/media/2018/FreeRTOS\\_Reference\\_Manual\\_V10.0.0.pdf](https://www.freertos.org/media/2018/FreeRTOS_Reference_Manual_V10.0.0.pdf)

- Important #includes:

```
#include "FreeRTOS.h" // Basic FreeRTOS functions
```

```
#include "task.h" // Task management functions like vTaskDelay
```

```
#include "semphr.h" // Semaphores
```

```
#include "queue.h" // Queues (next lab lecture)1
```

## 4. The FreeRTOS Interface

### ■ Task Creation:

`xTaskCreate(taskfun, taskname, stacksize, parameter, priority, handle);`

- `taskfun` – Pointer to the task function.
- `taskname` – The name of the task as a C string. Should be unique to aid debugging.
- `stacksize` – Size of the task stack. RTOS uses the task stack to pass parameters, create local variables, **and store the hardware context**.
  - ❑ Measured in words, not bytes.
  - ❑ Suggested; `configMINIMAL_STACK_SIZE + 100` (`configMINIMAL_STACK_SIZE` defaults to 90 words).
- `parameter` – Parameter to pass to the task
- `priority` – Priority number. Higher number == higher priority.
- `handle` – Pointer to a structure of type `TaskHandle_t` to store task information. Equivalent to `pthread_t` in pthreads.

## 4. The FreeRTOS Interface

### ■ Starting the RTOS:

`vTaskStartScheduler();`

- Starts the RTOS.
- Must be called after creating all the required tasks.

### ■ Example:

```
static void task(void *p) {  
    while(1) {  
        PRINTF("I am task %d\r\n", (int) p);  
        vTaskSuspend(NULL); // Stop the task  
    }  
}
```

## 4. The FreeRTOS Interface

### ■ Task creation example:

```
...
int tasknum=157;
// Create a task with priority 2
xTaskCreate(task, "task_1", configMINIMAL_STACK_SIZE+100,
            (void *) taskNum, 2, NULL);

// Create a higher priority task with priority 4
xTaskCreate(task, "task_2", configMINIMAL_STACK_SIZE+100,
            (void *) (taskNum+1), 4, NULL);

// Start the RTOS
vTaskStartScheduler();

...
```

## 4. The FreeRTOS Interface

### ■ Pausing a task:

`vTaskDelay(ticks);`

- ticks: Number of clock ticks to pause.
- Use `pdMS_TO_TICKS` to convert from milliseconds to ticks.
- Example:

```
vTaskDelay(pdMS_TO_TICKS(250)); // Pause for 250 ms
```

### ■ Suspending a task.

`vTaskSuspend(taskhandle);`

- taskhandle – Handle of task to suspend. Passing NULL will cause the caller to be suspended.
- Remains suspended until another task wakes it using `vTaskResume`.

## 4. The FreeRTOS Interface

### ■ Resuming a Task

`vTaskResume(handle)`

- `handle` : Handle of task to resume, of type `TaskHandle_t` (see `xTaskCreate`).
- Cannot be `NULL`.

### ■ Creating a Binary Semaphore

`xSemaphoreCreateBinary()`

- Creates a new binary semaphore with starting value of 0.
- Example:

```
SemaphoreHandle_t sema;  
sema = xSemaphoreCreateBinary();
```

## 4. The FreeRTOS Interface

### ■ Creating a Mutex

`xSemaphoreCreateMutex()`

- Creates a new binary semaphore with starting value of 1 instead of 0.
- Used for mutual exclusions. See Lab 6 for example.

### ■ Creating a Counting Semaphore

`xSemaphoreCreateCounting(max, init)`

- A counting semaphore can hold any non-negative value, not just 0 and 1.
- max: Maximum value the semaphore can hold.
- init: Starting value.

```
// Maximum value of 100, initial value of 0.  
sema = xSemaphoreCreateCounting(100, 0);
```

## 4. The FreeRTOS Interface

### ■ Taking a Semaphore

`xSemaphoreTake(sema, timeout)`

- `sema`: Semaphore to take. Blocks if `sema` is 0.
- `timeout`: Block for up to “timeout” ticks before exiting.
  - Specify `portMAX_DELAY` for maximum timeout ( $2^{32} - 1$  on the MCXC444).
- Returns `pdFALSE` if the call timed out, or `pdTRUE` if another task called `xSemaphoreGive` to unblock this task.

```
SemaphoreHandle_t sema;  
  
...  
if(xSemaphoreTake(sema, portMAX_DELAY) == pdTRUE) {  
    // Perform action.  
}
```

## 4. The FreeRTOS Interface

### ■ Giving a Semaphore:

`xSemaphoreGive(sema)`

- Unblocks a task that is blocked on sema if any. Otherwise increments sema.
- sema: Semaphore to give.

### ■ Giving a Semaphore Within an ISR:

`xSemaphoreGiveFromISR(sema, &hpx)`

- Used within ISRs to ensure context switch if a higher priority task is woken.
- sema: Semaphore to give.
- hpx: A variable of type `BaseType_t`:
  - Set to `pdTRUE` if the give unblocked a higher priority task than the one which was just interrupted by this ISR.

## 4. The FreeRTOS Interface

### ■ Yielding within an ISR

`portYIELD_FROM_ISR(hpw);`

- If `hpw` is `pdTRUE`, informs scheduler to switch to higher priority task when the ISR exits.

### ■ ISR Example:

```
SemaphoreHandle_t sema;
void portA_IRQHandler() {
    NVIC_ClearPendingIRQ(PORTA_IRQn);
    if(PORTA->ISFR & (1 << SW_PIN)) {
        BaseType_t hpw = pdFALSE;
        xSemaphoreGiveFromISR(sema, &hpw);
        portYIELD_FROM_ISR(hpw);
    }
    // Clear ISFR
    PORTA->ISFR |= (1 << SW_PIN);
}
```

---

## 5. Conclusion

- In this lab lecture we looked at how to design software to use an RTOS, and looked some FreeRTOS functions.
- Although the contents of this lecture are not examinable, you should know it well.
  - Very important for being a good computer engineer.